EMPLOYEE DATA CLEANING USING PYTHON

Employee Data Cleaning & Preprocessing Using Python

Prepared By: Ritesh Chaudhary

DECLARATION

I declare that this project documentation, titled "Employee Data Cleaning and Preprocessing Using Python", represents my understanding and practical implementation of data cleaning and preprocessing techniques using Python and its Pandas library.

The project demonstrates the process of identifying and handling duplicate records, missing values, unwanted characters, whitespace and inconsistent data types in an employee dataset.

ACKNOWLEDGEMENT

I would like to express my gratitude to my teachers and learning resources for providing the knowledge and guidance required to complete this practical project.

This project helped me develop practical knowledge of Python, Pandas and data preprocessing techniques. It also improved my understanding of the importance of data quality before performing further analysis.

ABSTRACT

Data cleaning and preprocessing are essential stages in data analysis. Real-world datasets often contain duplicate records, missing values, unwanted spaces, inconsistent text and incorrect data types. These problems can affect the accuracy and reliability of analytical results.

This project focuses on cleaning and preprocessing an employee dataset using Python and the Pandas library. The dataset is provided in CSV format and contains employee information such as employee ID, first name, last name, email, phone number, hire date, job ID, salary, commission percentage, manager ID and department ID.

The program implements several data cleaning techniques, including duplicate identification and removal, missing-value analysis, handling missing values using statistical methods, whitespace removal, character replacement and datatype conversion.

After applying these techniques, the dataset becomes more consistent and suitable for further data analysis, reporting and other data processing activities.

Keywords: Python, Pandas, Data Cleaning, Data Preprocessing, Missing Values, Duplicate Records, Feature Engineering, Data Analysis.

Contents

DECLARATION **1**

ACKNOWLEDGEMENT **1**

ABSTRACT **1**

1. INTRODUCTION **4**

2. PROJECT OBJECTIVES **4**

3. PROJECT SCOPE **4**

4. TOOLS AND TECHNOLOGIES **5**

5. DATASET DESCRIPTION **6**

5.1 Dataset Attributes 6

5.2 Initial Data Observation 6

6. SYSTEM REQUIREMENTS **7**

Hardware Requirements 7

Software Requirements 7

Library Installation 7

7. METHODOLOGY **7**

8. PROGRAM IMPLEMENTATION **8**

8.1 Importing Libraries and Loading Data 8

8.2 Initial Data Inspection 8

8.3 Duplicate Data Identification and Removal 9

8.4 Missing Data Identification 10

8.5 Handling Missing Values 11

A. Removing Missing Data 11

B. Filling Missing Values Using Mean 12

C. Filling Missing Values Using Mode 13

8.6 Removing Unwanted Data 14

A. Identifying String Columns 14

B. Checking Unique Values 15

C. Removing Whitespace 16

D. Replacing Unwanted Characters 16

8.7 Datatype Conversion (Typecasting) 17

A. Checking Existing Datatypes 17

B. Converting String to Datetime 17

C. Converting Float to Integer 18

D. Final Datatype Inspection 18
9. PROGRAM DEMONSTRATION 19
Step 1: Load the Employee Dataset 19
Step 2: Identify Duplicate Employees 19
Step 3: Remove Duplicate Records 19
Step 4: Identify Missing Values 19
Step 5: Fill Missing Values 20
Step 6: Clean Text Data 20
Step 7: Convert Datatypes 20
Step 8: Validate the Final Dataset 20
10. RESULTS AND OBSERVATIONS 20
10.1 Summary of Data Cleaning 20
10.2 Final Dataset Structure 21
10.3 Benefits Achieved 21
11. LIMITATIONS 21
12. FUTURE ENHANCEMENTS 21
13. CONCLUSION 23
14. REFERENCES 23

1. INTRODUCTION

Data is an important resource for organisations because it supports business decisions, reporting and operational activities. However, raw data collected from different sources is not always accurate, complete or consistent.

For example, an employee dataset may contain duplicate employee records, missing commission values, unnecessary whitespace in names, or dates stored as text rather than proper datetime values.

Using such data without proper preparation can produce inaccurate analytical results.

Data cleaning is the process of identifying and correcting or handling these data quality issues. Data preprocessing prepares the cleaned information for further analysis.

In this project, Python and the Pandas library are used to perform practical data cleaning operations on an employee dataset.

The program follows a systematic approach to inspect the dataset, identify data quality problems, apply appropriate cleaning techniques and validate the resulting data.

2. PROJECT OBJECTIVES

The main objectives of this project are:

- To understand the importance of data cleaning and preprocessing.
- To import and inspect CSV datasets using Pandas.
- To identify and remove duplicate employee records.
- To identify missing values in different columns.
- To handle missing numerical values using statistical techniques.
- To handle missing categorical or discrete values using mode.
- To remove unnecessary whitespace from string columns.
- To identify and replace unwanted characters in text data.
- To convert columns into appropriate data types.
- To validate the cleaned dataset before further analysis.
- To develop practical Python and data manipulation skills.

3. PROJECT SCOPE

This project focuses on the preparation and improvement of an employee dataset.

The scope includes:

Data Inspection: Understanding the dataset structure, columns, number of records and data types.

Duplicate Handling: Identifying duplicate employee records using the employee ID and removing repeated entries.

Missing Value Handling: Identifying null values and applying suitable strategies to handle them.

Text Cleaning: Removing unnecessary spaces and demonstrating character replacement in string columns.

Datatype Conversion: Converting date columns into datetime format and numerical columns into appropriate numeric types.

Data Validation: Checking the final dataset to confirm that the implemented cleaning operations have been applied correctly.

The project focuses on data preparation rather than building dashboards, predictive models or performing advanced statistical analysis.

4. TOOLS AND TECHNOLOGIES

The following technologies and libraries are used in this project.

Technology	Purpose
Python	Main programming language
Pandas	Data manipulation and cleaning
NumPy	Numerical operations and statistical support
CSV	Source data format
Jupyter Notebook	Writing and executing Python code

Python: Python is used as the primary programming language because of its simplicity and extensive support for data processing.

Pandas: Pandas provides DataFrame structures and functions for handling missing values, duplicate records, text operations and datatype conversions.

NumPy: NumPy supports numerical computing and works alongside Pandas for mathematical operations.

Jupyter Notebook: Jupyter Notebook provides an interactive environment where code can be executed step by step and its outputs can be inspected.

5. DATASET DESCRIPTION

The project uses an employee dataset named Employees.csv.

The dataset contains employee-related information, including personal identifiers, employment details, salary information and organisational relationships.

Initial dataset information:

Description	Details
Source file	Employees.csv
Initial records	111
Total columns	11
File format	CSV
Data structure	Tabular
Unique employee records after duplicate removal	107

5.1 Dataset Attributes

The dataset contains the following columns:

Column Name	Description
EMPLOYEE_ID	Unique identifier assigned to an employee
FIRST_NAME	Employee's first name
LAST_NAME	Employee's last name
EMAIL	Employee email identifier
PHONE_NUMBER	Employee contact number
HIRE_DATE	Employee joining date
JOB_ID	Identifier representing the employee's job role
SALARY	Employee salary
COMMISSION_PCT	Employee commission percentage
MANAGER_ID	Identifier of the employee's manager
DEPARTMENT_ID	Identifier of the employee's department

5.2 Initial Data Observation

The initial dataset contains 111 records and 11 columns.

During the initial inspection, the following issues were identified:

- Repeated employee records.

- Missing values in commission percentage.
- Missing values in manager and department identifiers.
- Date values stored in string format.
- Potential unwanted whitespace and inconsistent text values requiring inspection.

These issues are addressed through the data cleaning process.

6. SYSTEM REQUIREMENTS

Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Sufficient storage for Python, libraries and dataset files.

Software Requirements

- Python 3.x
- Jupyter Notebook or another Python IDE.
- Pandas library.
- NumPy library.
- Employees.csv dataset.

Library Installation

The required libraries can be installed using:

pip install pandas numpy jupyter

7. METHODOLOGY

The project follows a sequential data cleaning and preprocessing methodology.

The main stages are:

1. Data loading.
2. Initial data inspection.
3. Duplicate identification and removal.
4. Missing-value identification.
5. Missing-value handling.
6. Unwanted text and whitespace removal.
7. Datatype conversion.
8. Final data validation.

Each stage is performed to improve the quality and consistency of the employee dataset.

8. PROGRAM IMPLEMENTATION

8.1 Importing Libraries and Loading Data

The first step is to import the required Python libraries and load the CSV file into a Pandas DataFrame.

Code:

```
import pandas as pd
import numpy as np

df = pd.read_csv('Employees.csv')

df.head()
```

Explanation:

- import pandas as pd imports the Pandas library.
- import numpy as np imports NumPy.
- pd.read_csv() reads the employee CSV file.
- The loaded dataset is stored in the DataFrame variable df.
- df.head() displays the first five records.

A DataFrame is a two-dimensional tabular data structure containing rows and columns.

8.2 Initial Data Inspection

Before modifying the dataset, it is important to understand its structure and identify potential data quality problems.

Some useful Pandas functions are:

```
df.head()
df.tail()
df.shape
df.info()
df.isnull().sum()
df.describe()
```

Explanation:

- head() displays the first five records.

- tail() displays the last five records.
- shape returns the number of rows and columns.
- info() provides column names, non-null counts and datatypes.
- isnull().sum() calculates missing values in each column.
- describe() provides descriptive statistics for numerical columns.

These operations help establish the initial condition of the dataset before cleaning.

8.3 Duplicate Data Identification and Removal

Duplicate records occur when the same information appears more than once in a dataset.

In this project, EMPLOYEE_ID is used to identify repeated employee records because it represents the employee's unique identifier.

Step 1: Identify Duplicate Records

```
duplicate_data = df[
   df.duplicated(subset='EMPLOYEE_ID')
]

duplicate_data
```

Explanation:

- duplicated() identifies repeated values.
- subset='EMPLOYEE_ID' checks duplication based on employee ID.
- By default, the first occurrence is treated as the original record, while subsequent occurrences are identified as duplicates.

The initial dataset contained four repeated employee records.

Step 2: Verify a Specific Employee ID

```
emp_id = int(input('Enter emp id: '))

data = df[df['EMPLOYEE_ID'] == emp_id]

data
```

Explanation:

This code accepts an employee ID from the user and filters the dataset to display all records associated with that ID.

It provides a simple way to verify whether a particular employee appears more than once.

Step 3: Remove Duplicate Records

```
df.drop_duplicates(
   subset='EMPLOYEE_ID',
   inplace=True
)
```

Explanation:

The drop_duplicates() function removes repeated records based on the employee ID.

The inplace=True argument applies the modification directly to the existing DataFrame.

Step 4: Verify Duplicate Removal

```
duplicate_data = df[
   df.duplicated(subset='EMPLOYEE_ID')
]

duplicate_data
```

The result should contain no duplicate records based on EMPLOYEE_ID.

Observation: The dataset was reduced from 111 records to 107 records after removing four duplicate entries.

8.4 Missing Data Identification

Missing values occur when information is unavailable or not recorded in a particular column.

Identifying missing values is important because incomplete data can affect calculations and further analysis.

Code:

```
df.isnull().sum()
```

Initial observation:

Column	Missing Values
COMMISSION_PCT	72
MANAGER_ID	1
DEPARTMENT_ID	1
Other columns	0

The dataset contains missing values in three columns.

Different strategies are required depending on the nature of the missing information.

8.5 Handling Missing Values

Missing values can be handled using different techniques, including:

- Removing rows or columns.
- Filling missing values with the mean.
- Filling missing values with the median.
- Filling missing values with the mode.
- Using predefined or custom values.

The appropriate method depends on the column's data type, distribution, business meaning and the reason the values are missing.

A. Removing Missing Data

Pandas provides two important functions:

dropna(): Removes rows or columns based on missing-value conditions.

drop(): Removes specified rows or columns regardless of whether they contain missing values.

Example:

```
df.dropna(
   axis=0,
   how='all',
   inplace=True
)
```

This removes rows where all columns contain missing values.

Another example:

```
df.drop(columns=['EMAIL'])
```

This removes the specified EMAIL column from the returned DataFrame. Without inplace=True or reassignment, the original DataFrame remains unchanged.

Important: Removing every row containing at least one missing value is not always appropriate. In this dataset, doing so could remove a large amount of useful information because the commission column contains many missing values.

B. Filling Missing Values Using Mean

The mean is the arithmetic average of numerical values.

It can be used to fill missing numerical data when it is suitable for the distribution and meaning of the variable.

In this project, the skewness of the commission column is examined before selecting an imputation method.

Step 1: Calculate Skewness

```
df['COMMISSION_PCT'].skew()
```

The observed skewness value is approximately:

0.0966

This indicates that the observed commission values have relatively low skewness.

Step 2: Calculate Mean

```
comm_mean = round(
   df['COMMISSION_PCT'].mean(),
   2
)

comm_mean
```

Observed mean:

0.22

Step 3: Fill Missing Values

```
df['COMMISSION_PCT'] = (
   df['COMMISSION_PCT'].fillna(comm_mean)
)
```

Explanation:

- mean() calculates the average commission value.
- round() rounds the result to two decimal places.
- fillna() replaces missing values with the calculated mean.
- The result is assigned back to the original column.

Observation: Missing commission values are replaced with the calculated mean of 0.22.

This approach assumes that the missing commission values can reasonably be represented by the observed average. In a real business dataset, the meaning of a missing commission value should be checked before applying this assumption.

C. Filling Missing Values Using Mode

The mode represents the most frequently occurring value in a dataset.

It is useful for categorical and discrete values, including identifiers when an appropriate imputation decision has been established.

In this project, the manager and department columns are examined.

Step 1: Calculate Mode

```
mgr_mode = df['MANAGER_ID'].mode()

dept_mode = df['DEPARTMENT_ID'].mode()

print(mgr_mode)
print(dept_mode)
```

Observed most frequent values:

- MANAGER_ID: 100
- DEPARTMENT_ID: 50

Step 2: Select a Single Mode Value

```
mgr_mode = df['MANAGER_ID'].mode()[0]

dept_mode = df['DEPARTMENT_ID'].mode()[0]
```

The [0] index selects the first value returned by the mode function.

This is important because Pandas can return multiple modes when more than one value has the same highest frequency.

Step 3: Fill Missing Values

```
df['MANAGER_ID'] = (
   df['MANAGER_ID'].fillna(mgr_mode)
)

df['DEPARTMENT_ID'] = (
```

```
    df['DEPARTMENT_ID'].fillna(dept_mode)
)
```

Explanation:

The missing manager and department values are replaced using their respective most frequent values.

Step 4: Validate Missing Values

df.isnull().sum()

Expected result after applying the three imputation operations:

```
EMPLOYEE_ID       0
FIRST_NAME        0
LAST_NAME         0
EMAIL             0
PHONE_NUMBER      0
HIRE_DATE         0
JOB_ID            0
SALARY            0
COMMISSION_PCT    0
MANAGER_ID        0
DEPARTMENT_ID     0
```

The missing-value count is now zero across all columns.

Important consideration: The mode-based replacement is a demonstration of statistical imputation. In actual employee records, manager and department identifiers should preferably be validated against organisational relationships rather than automatically assigning the most frequent identifier.

8.6 Removing Unwanted Data

Text columns may contain unnecessary whitespace or unwanted characters that can create inconsistencies.

Pandas provides string methods to clean text data.

A. Identifying String Columns

object_dtype = df.select_dtypes('str')

object_dtype

Explanation:

select_dtypes() selects columns matching the specified datatype.

In this project, string columns are selected for text cleaning.

Date-like columns such as HIRE_DATE should be handled separately because they require conversion into datetime format.

B. Checking Unique Values

The unique() function displays distinct values in a column.

df['FIRST_NAME'].unique()

This helps inspect the values and identify possible inconsistencies.

Multiple columns can also be checked using a loop.

```
columns = [
    'FIRST_NAME',
    'LAST_NAME',
    'EMAIL',
    'JOB_ID'
]

for col in columns:
    print(col)
    print('-' * 8)

    unique_data = df[col].unique()

    print(unique_data)
    print('\n')
```

Explanation:

The loop iterates through the specified columns and displays their unique values.

This makes it easier to inspect multiple text columns without writing separate statements for each column.

C. Removing Whitespace

The strip() method removes whitespace from both the beginning and end of a string.

Other related methods include:

- strip(): Removes whitespace from both sides.
- lstrip(): Removes whitespace from the left side.
- rstrip(): Removes whitespace from the right side.

Example:

```
df['FIRST_NAME'] = (
   df['FIRST_NAME'].str.strip()
)
```

To apply the operation to multiple columns, a reusable function is created.

```
def removeWhitespace(dataframe, col_names):

   for col in col_names:
      dataframe[col] = (
         dataframe[col].str.strip()
      )
```

Function execution:

```
removeWhitespace(df, columns)
```

Explanation:

The function accepts a DataFrame and a list of column names.

It loops through each selected column and removes unnecessary whitespace using the str.strip() method.

This reduces repeated code and makes the cleaning process reusable.

D. Replacing Unwanted Characters

The str.replace() method can replace specific characters or text within string values.

Example:

```
df['FIRST_NAME'] = (
```

```
    df['FIRST_NAME'].str.replace(
        'ven',
        '***',
        regex=False
    )
)
```

This replaces the text ven with *** wherever it occurs in the FIRST_NAME column.

Note: This is an illustrative example of the replacement function. It is not a recommended modification to actual employee names. In a production cleaning process, replacements should only be applied when they correct an identified data quality issue.

8.7 Datatype Conversion (Typecasting)

Correct datatypes are important because they determine how data can be processed and analysed.

For example, dates stored as strings cannot be directly used for all datetime-based operations.

CSV files commonly store date values as text, so conversion is required after loading when the data is recognised as a string.

A. Checking Existing Datatypes

df.info()

Before conversion, the HIRE_DATE column is stored as a string.

Other columns include:

- EMPLOYEE_ID: Integer
- SALARY: Integer
- COMMISSION_PCT: Float
- MANAGER_ID: Float
- DEPARTMENT_ID: Float

B. Converting String to Datetime

```
df['HIRE_DATE'] = pd.to_datetime(
    df['HIRE_DATE'],
    format='mixed'
)
```

Explanation:

- pd.to_datetime() converts date values into Pandas datetime format.
- format='mixed' allows Pandas to infer individual date formats where supported.
- The converted result is assigned back to the HIRE_DATE column.

After conversion, dates are represented in a standard format such as:

2003-06-17
2005-09-21
2001-01-13

This allows further operations such as extracting the year, month and day, calculating employee service duration and analysing hiring trends.

C. Converting Float to Integer

After handling missing values, the DEPARTMENT_ID column contains numeric values represented as floating-point numbers.

Since department identifiers are whole numbers, they can be converted into integers.

Code:

```
df['DEPARTMENT_ID'] = (
    df['DEPARTMENT_ID'].astype(int)
)
```

Explanation:

- astype(int) converts the selected column into an integer datatype.
- The converted result is assigned back to the DataFrame.

This removes unnecessary decimal representations from department identifiers.

D. Final Datatype Inspection

df.info()

After conversion, the expected datatypes include:

Column	Final Datatype
EMPLOYEE_ID	Integer
FIRST_NAME	String
LAST_NAME	String

EMAIL	String
PHONE_NUMBER	String
HIRE_DATE	Datetime
JOB_ID	String
SALARY	Integer
COMMISSION_PCT	Float
MANAGER_ID	Float
DEPARTMENT_ID	Integer

The manager identifier remains a float because the original column was not converted to an integer in the demonstrated implementation.

9. PROGRAM DEMONSTRATION

This section provides a practical demonstration of how the program works from the user's perspective.

Step 1: Load the Employee Dataset

The program reads Employees.csv and stores its contents in a Pandas DataFrame.

The first five records are displayed to understand the dataset.

Expected initial dataset size: 111 rows and 11 columns.

Step 2: Identify Duplicate Employees

The program checks employee IDs and identifies repeated records.

For example, entering employee ID 203 displays the matching records.

This allows the user to inspect the repeated employee information before removing duplicates.

Step 3: Remove Duplicate Records

The program removes repeated employee records based on EMPLOYEE_ID.

Result: Four duplicate records are removed, leaving 107 records.

Step 4: Identify Missing Values

The program calculates missing values for every column.

The user can identify which columns require further treatment.

Step 5: Fill Missing Values

The program applies different strategies:

- Commission percentage: Mean.
- Manager ID: Mode.
- Department ID: Mode.

The missing-value count is checked again after imputation.

Step 6: Clean Text Data

The program selects string columns, checks unique values, removes unnecessary whitespace and demonstrates text replacement.

Step 7: Convert Datatypes

The HIRE_DATE column is converted into datetime format.

The DEPARTMENT_ID column is converted into integer format.

Step 8: Validate the Final Dataset

The program uses info() and isnull().sum() to inspect the final DataFrame.

The final dataset contains 107 records and 11 columns, with no missing values reported after the implemented imputation operations.

10. RESULTS AND OBSERVATIONS

After completing the data cleaning operations, the following results were observed.

10.1 Summary of Data Cleaning

Cleaning Operation	**Initial Condition**	**Final Result**
Duplicate removal	111 records	107 records
Commission missing values	72	0
Manager missing values	1	0
Department missing values	1	0
Whitespace cleaning	Text columns inspected	Strip operation applied
Date conversion	String	Datetime
Department ID conversion	Float	Integer

10.2 Final Dataset Structure

The final DataFrame contains:

- 107 employee records.
- 11 columns.
- No duplicate employee IDs based on the selected subset.
- No missing values after the implemented imputation operations.
- A datetime datatype for HIRE_DATE.
- An integer datatype for DEPARTMENT_ID.

10.3 Benefits Achieved

The project demonstrates several important benefits of data cleaning:

1. Improved data consistency through duplicate identification and removal.
2. Better completeness through missing-value handling.
3. Improved text quality through whitespace cleaning.
4. More appropriate datatypes for further processing.
5. Better understanding of Pandas DataFrame operations.
6. Preparation of structured data for future analysis and reporting.

11. LIMITATIONS

Although the project demonstrates important data cleaning techniques, some limitations remain.

- The project uses a relatively small employee dataset.
- Missing values are handled using basic statistical imputation methods.
- Replacing missing manager and department identifiers with their modes may not always represent the correct organisational relationships.
- Mean-based imputation may change the original statistical distribution of the commission column.
- The program does not include advanced outlier detection.
- The program does not perform automated data quality reporting.
- The final cleaned dataset is not yet exported to a separate output file in the demonstrated implementation.

These limitations provide opportunities for further improvement.

12. FUTURE ENHANCEMENTS

The project can be extended with additional functionality to improve its usefulness in practical data analysis environments.

Possible enhancements include:

1. Automated Data Quality Report

Generate a report showing duplicate counts, missing values, datatypes and other data quality issues.

2. Outlier Detection

Implement statistical techniques such as the Interquartile Range (IQR) method to identify unusual numerical values.

3. Feature Engineering

Create additional columns by combining or extracting existing information.

Examples:

- Combine FIRST_NAME and LAST_NAME into FULL_NAME.
- Extract hiring year from HIRE_DATE.
- Calculate employee service duration.
- Group employees based on salary ranges.

4. Data Export

Export the cleaned dataset into CSV or Excel format using Pandas.

Example:

```
df.to_csv(
    'Cleaned_Employees.csv',
    index=False
)
```

5. Reusable Cleaning Functions

Organise the cleaning operations into reusable functions to improve maintainability and reduce repeated code.

6. Data Visualisation

Use Matplotlib, Seaborn or Power BI to visualise employee distributions, salary information and department-level statistics.

13. CONCLUSION

This project demonstrates the practical implementation of employee data cleaning and preprocessing using Python and the Pandas library.

The program identifies and removes duplicate employee records, examines missing values, applies statistical imputation techniques, removes unnecessary whitespace, demonstrates text replacement and converts columns into appropriate datatypes.

Through this project, practical knowledge of DataFrames, data manipulation, statistical methods, string operations, loops, reusable functions and datatype conversion has been developed.

The project also highlights an important concept in data analytics: the quality of analytical results depends heavily on the quality of the data used.

Overall, this project provides a foundation for more advanced data analysis, data visualisation, reporting and data engineering activities.

14. REFERENCES

1. Pandas Documentation. (n.d.). *Pandas: Python Data Analysis Library*. Available at: https://pandas.pydata.org/docs/
2. Python Software Foundation. (n.d.). *Python Documentation*. Available at: https://docs.python.org/3/
3. NumPy Documentation. (n.d.). *NumPy: The Fundamental Package for Scientific Computing with Python*. Available at: https://numpy.org/doc/
4. Pandas Documentation. (n.d.). *Working with Missing Data*. Available at: https://pandas.pydata.org/docs/user_guide/missing_data.html
5. Pandas Documentation. (n.d.). *Text Data*. Available at: https://pandas.pydata.org/docs/user_guide/text.html

---

END OF PROJECT DOCUMENTATION